Day - 14
Tuple
in Python

SKILL COURSE

A.I. Powered 30 Days Python Micro Course By Satish Dhawale (Microsoft Certified Trainer)

What is a Tuple?

- A **tuple** is like a **permanent list** whose values **cannot be changed**.
- Think of it like:
- 👉 Your Aadhaar details
  👉 Date of birth
  👉 Country codes
  👉 Fixed coordinates (latitude, longitude)
- Once created → **You cannot modify it**.

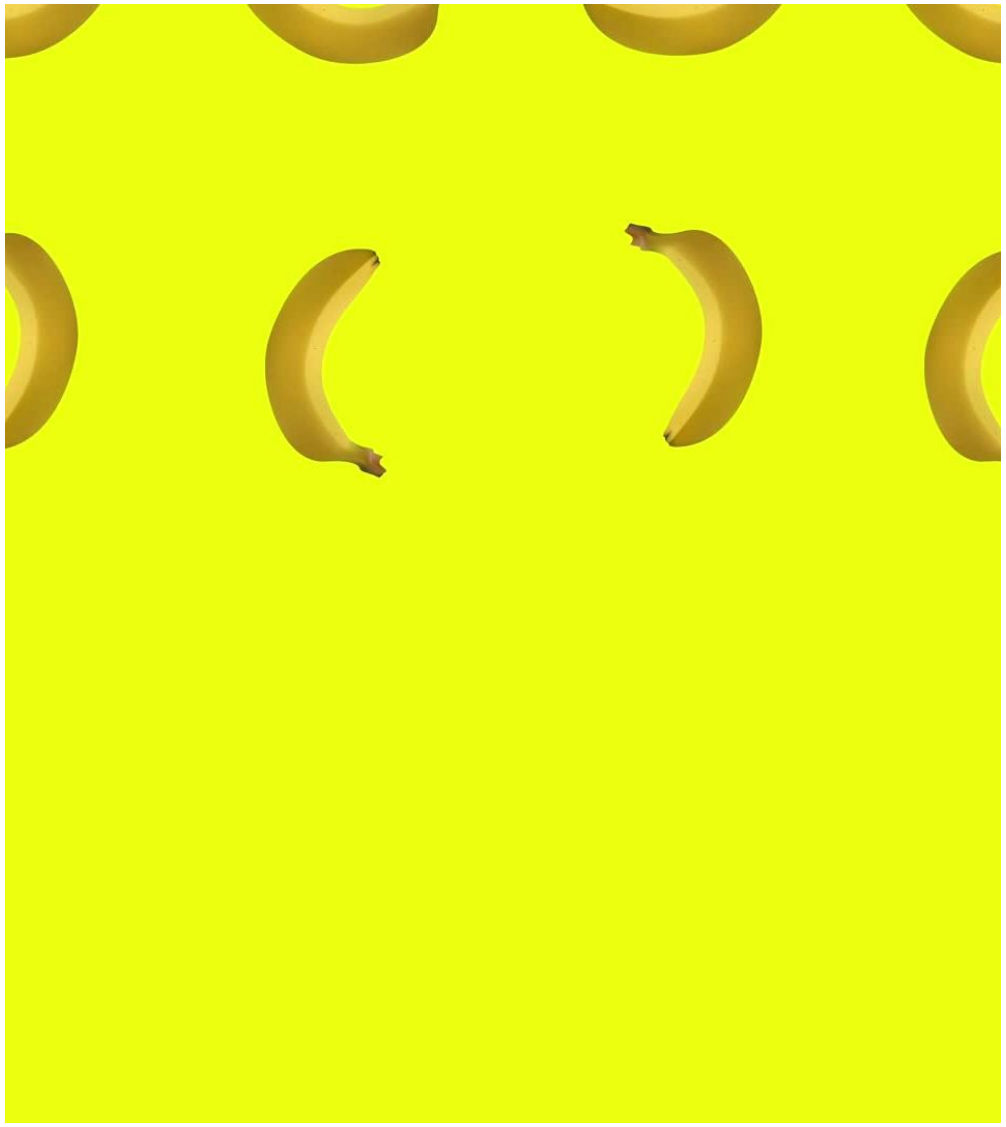
- ✓ **Technical Definition**
- A tuple is an **ordered, immutable collection** that stores multiple items inside **( )**.
- Example: ("Apple", "Banana", "Mango")

Why Tuples Are Important for Data Analysts?

- Store **fixed values**
- Prevent accidental modifications
- Store **coordinates** (latitude, longitude)
- Store values that should not change in code
- Faster than lists (important in large data)
- Used in CSV rows returned by DB fetch operations
- Used in Pandas .itertuples()

Tuple vs List (Very Important Slide)

Feature	List	Tuple
Brackets	[]	()
Mutable	✓ Yes	❌ No
Speed	Slower	Faster
Use Case	Changing data	Fixed data
Examples	Student names	Lat/Long



Creating and Accessing Tuple Items

- fruits = ("Apple", "Banana", "Mango")

- print(fruits[0]) # Apple
- print(fruits[-1]) # Mango

- numbers = (10, 20, 30, 40, 50)
- print(numbers[1:4]) # (20, 30, 40)

Practical

- Tuples Are Immutable (Cannot Change)
  - fruits = ("Apple", "Banana", "Mango")
  - # fruits[1] = "Orange" # ❌ Error

- Looping Through Tuples
  - colors = ("Red", "Blue", "Green")

  - for c in colors:
  - print(c)

Practical

- Tuple Length
  - print(len(colors))
- Concatenation of Tuples’
  - a = (1, 2)
  - b = (3, 4)
  - c = a + b
  - print(c) # (1,2,3,4)

Packing & Unpacking (Very Important)

```
data = ("Laptop", 45000, "Black")

product, price, color = data
print(product)
print(price)
print(color)
```

Nesting Tuples Inside Lists (Real Use Case)

```
employees = [
    ("E101", "Rohit", "Pune"),
    ("E102", "Sneha", "Mumbai"),
    ("E103", "Ayesha", "Delhi")
]

for eid, name, city in employees:
    print(name, "from", city)
```